

INSuRE Spring 2023 Final Presentation

Malware Similarity

Sponsor: MITRE

Technical Director
Frank Posluszny(malwarefrank@mitre.org)

Faculty
Brian Ricks(bwr031000@utdallas.edu)



Team

Kushan Singh (kxs200085@utdallas.edu)

Computer Science, UTD

Haoyan Li (hxl210024@utdallas.edu)

Computer Science, UTD

Jerry Teng (jerry.teng@utdallas.edu)

Computer Science, UTD

Payton Harmon (pnh170000@utdallas.edu)

Computer Science, UTD

Surekha Kumari (sxx210055@utdallas.edu)

Computer Engineering, UTD



Motivation

- Detection and prevention of malware attacks.
- Identifying source and origin of malwares.
- Understanding malware evolution.



Goals

Studying string similarities in malware analysis:

- Identifying common malware families.
- Analyzing malware variants.
- Understanding malware behavior.



Literature Review

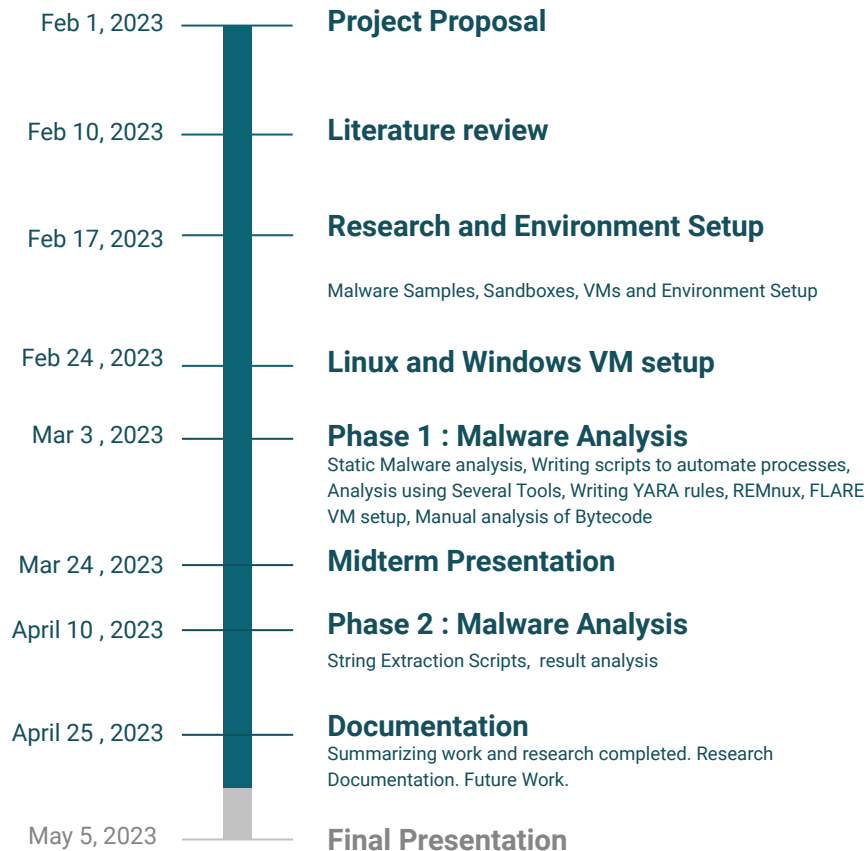
Malware Similarity Analysis: An adversary perspective - Frank Posluszny, March 2020



Figure 1 : Malware Development Life Cycle (simplified)



Timeline





Progress Overview

- Analyzed bytecode of tools that accept command line-arguments
 - Looking for patterns that indicate a program accepts command-line arguments
- Developed a script to perform large-scale information gathering for portable executable files
 - Provides output in CSV format, which can be ingested by machine learning models or other automated tools



Current Malware Samples

Name	Functionality	Heuristic Found/Rule exists
AdFind	Query Active Directory	No
NirSoft	Extract passwords saved in browser	Yes
Onhat	Network proxy	Yes
Ransomware	assorted ransomwares	No
RemCom	Remote Administration Tool	Yes
Upchurch Group 2	Network proxy	Yes
X-tunnel	Network proxy	No



Bytecode Analysis

- Reverse engineering: Given an binary executable file, reconstruct the original source code or program functionality
 - combination of static and dynamic analysis
 - Ghidra, IDA on REMnux analysis machine, OllyDbg on victim Flare VM
- Challenges:
 - Different compilers can produce different looking assembly code
 - Higher optimization levels can create nonstandard assembly code
 - Intentional low-level obfuscation techniques are a whole rabbit hole
- Goal: Can we create some rules or heuristics to detect whether a program accepts command line arguments?
- Main focus is on C/C++ Windows Portable Executable (PE) files



Heuristics Phase 1

- Analysis of pre-main initialization actions
- Steps:
 - Find the Windows API function that invokes command-line parsing and loads command-line string into memory
 - `__getmainargs`, `__wincmdln`, `GetCommandLineA/W`, etc
 - Track the location of the command-line string until main is called
 - Note the way that argv is passed to main (stack or register based)
- Why not just assume standard calling convention?
 - compiler differences
 - intentional obfuscation
 - manual assembly programming

NirSoft WebBrowserPassView

00401C1E	> 50	PUSH EAX	Arg4
00401C1F	· 56	PUSH ESI	Arg3, argv
00401C20	· 57	PUSH EDI	Arg2
00401C21	· 57	PUSH EDI	ModuleName
00401C22	· FF15 90F14500	CALL DWORD PTR DS:[<&KERNEL32.GetModuleHandleA>]	Arg1
00401C28	· 50	PUSH EAX	WebBrowserPassView.00446E80, main
00401C29	· E8 52520400	CALL 00446E80	

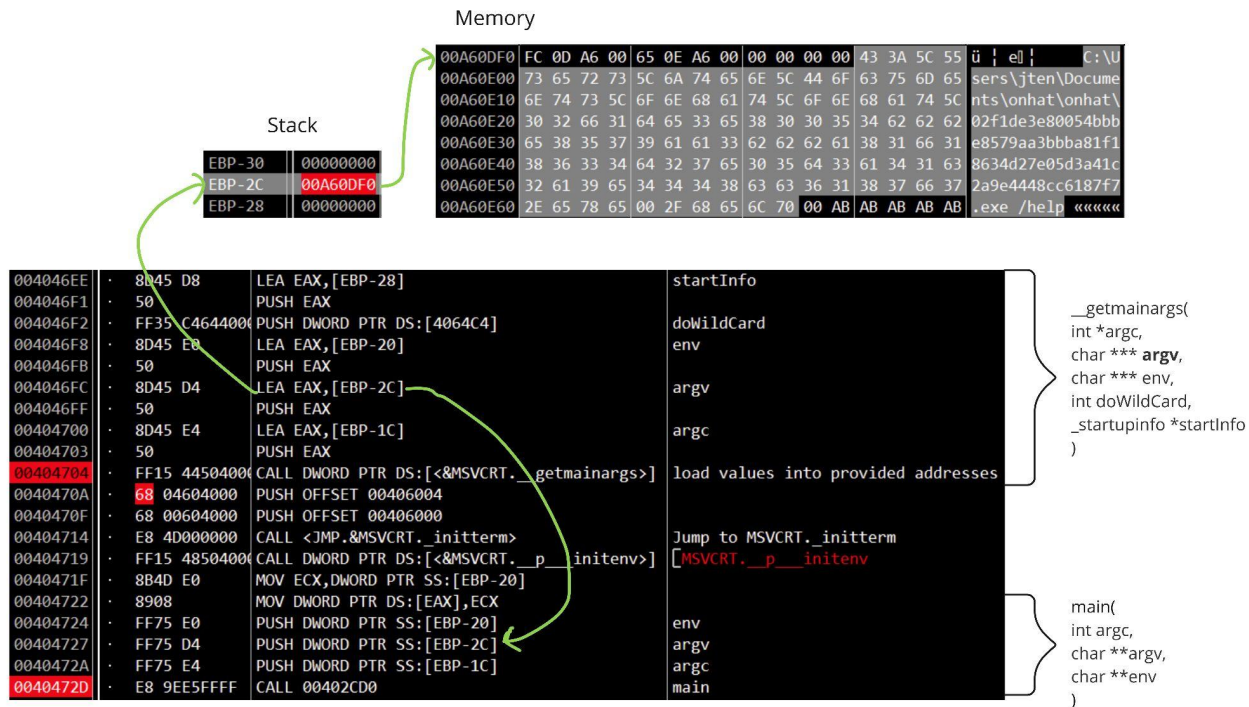
ESP ==>	00400000	@	Arg1 = WebBrowserPassView.<STRUCT IMAGE_DOS_HEADER>
ESP+4	00000000		Arg2 = 0
ESP+8	00712116	!lq	Arg3 = UNICODE "/help"
ESP+C	0000000A		Arg4 = 0A

Stack

Local Variables
EBP
Return Address
argc
*argv
*env

Expected
stack during
main

Heuristics Phase 1

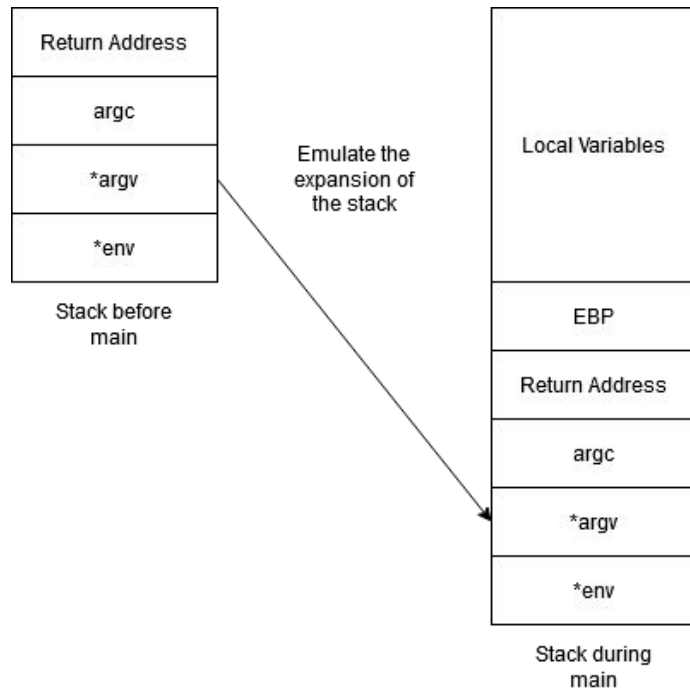


02f1de3e80054bbbe8579aa3bbba81f18634d27e05d3a41c2a9e4448cc6187f7 (ONHAT)



Heuristics Phase 2

- As the main function performs its function prologue, maintain knowledge of the location of argv
- For stack-based calling conventions, emulate the stack allocation operations so we can tell when argv is read relative to ESP or EBP



Heuristics Phase 2

```
main XREF
00402cd0 81 ec b0... SUB ESP, 0x2b0
00402cd6 33 c0 XOR EAX, EAX
00402cd8 8b 8c 24... MOV ECX, dword ptr [ESP + argc]
00402cdf 89 44 24... MOV dword ptr [ESP + local_278], EAX
00402ce3 89 44 24... MOV dword ptr [ESP + local_264], EAX
00402ce7 89 44 24... MOV dword ptr [ESP + local_25c], EAX
00402ceb 89 44 24... MOV dword ptr [ESP + local_258], EAX
00402cef 89 44 24... MOV dword ptr [ESP + local_27c], EAX
00402cf3 89 44 24... MOV dword ptr [ESP + local_29c], EAX
00402cf7 89 44 24... MOV dword ptr [ESP + local_260], EAX
00402cfb 89 44 24... MOV dword ptr [ESP + local_280], EAX
00402cff 53 PUSH EBX
00402d00 b0 3a MOV AL, 0x3a
00402d02 55 PUSH EBP
00402d03 56 PUSH ESI
00402d04 88 84 24... MOV byte ptr [ESP + local_20b], AL
00402d0b 88 84 24... MOV byte ptr [ESP + local_208], AL
00402d12 88 84 24... MOV byte ptr [ESP + local_205], AL
00402d19 88 84 24... MOV byte ptr [ESP + local_203], AL
00402d20 88 84 24... MOV byte ptr [ESP + local_201], AL
00402d27 88 84 24... MOV byte ptr [ESP + local_1ff], AL
00402d2e 88 84 24... MOV byte ptr [ESP + local_1fd], AL
00402d35 88 84 24... MOV byte ptr [ESP + local_1fb], AL
00402d3c 88 84 24... MOV byte ptr [ESP + local_1f9], AL
00402d43 57 PUSH EDI
00402d44 8b bc 24... MOV EDI, dword ptr [ESP + argv]
00402d4b 8d 84 24... LEA EAX=>local_20c, [ESP + 0xb4]
```

Initializing local variables, etc.

```
LAB_00446eaa XREF[1]
00446eaa e8 51 aa... CALL FUN_00441900
00446eaf 68 01 80... PUSH 0x8001
00446eb4 ff 15 f8... CALL dword ptr [->KERNEL32.DLL::SetError
00446eba e8 21 ac... CALL FUN_00441ae0
00446ebf 8d 7c 24... LEA EDI=>local_276c, [ESP + 0x2c]
00446ec3 e8 98 67... CALL FUN_0044d660
00446ec8 8d 44 24... LEA EAX=>local_2738, [ESP + 0x60]
00446ecc e8 9f fa... CALL FUN_00446970
00446ed1 8b 4d 10 MOV ECX, dword ptr [EBP + argv]
00446ed4 51 PUSH ECX
00446ed5 8b d7 MOV EDX, EDI
00446ed7 8b c7 MOV EAX, EDI
00446ed9 52 PUSH EDX
00446eda 89 84 24... MOV dword ptr [ESP + local_2094], EAX
00446ee1 e8 ea 68... CALL FUN_0044d7d0
```

NirSoft WebBrowserPassView: Access argv using EBP

ONHAT: Access argv using ESP



Heuristics Phase 3

- Detect any read references to the location of argv that may indicate it is being used
- Example:
 - Address of argv is loaded from stack into a register and pushed onto the stack as the argument for an argument parsing function
 - Register containing argv address is loaded into another register intended for callee

ONHAT: Argv is referenced and pushed onto stack

```
00402d43 57      PUSH     EDI
00402d44 8b bc 24... MOV     EDI, dword ptr [ESP + argv]
00402d4b 8d 84 24... LEA     EAX=>local_20c, [ESP + 0xb4]
00402d52 50      PUSH     EAX
00402d53 57      PUSH     EDI
00402d54 51      PUSH     ECX
00402dbf e8 fc 16... CALL    doSomethingWithCommandLineString
```

0019FC34	00000002		Arg1 = 2
0019FC38	00A00DF0	ð	Arg2 = 0A00DF0
0019FC3C	0019FCF4	öü	Arg3 = ASCII "b:ef:hl:o:0:p:P:s:t:"

Stack

NirSoft WBPV: Argv is referenced and pushed onto stack

```
00446ed1 8b 4d 10    MOV     ECX, dword ptr [EBP + argv]
00446ed4 51          PUSH    ECX
00446ed5 8b d7      MOV     EDX, EDI
00446ed7 8b c7      MOV     EAX, EDI
00446ed9 52          PUSH    EDX
00446eda 89 84 24... MOV     dword ptr [ESP + local_2094], EAX
00446ee1 e8 ea 68... CALL    FUN_0044d7d0
```

0019D730	0019D764	d×	Arg1 = 19D764
0019D734	006F21EC	ì!o	Arg2 = UNICODE "/help"

Stack



Heuristics Summary

- Can be hard to define exact patterns due to compiler differences, intentional obfuscation, system differences, library differences, etc.
- Created a general heuristic for how to track the usage of argv
- Can be used in conjunction with other heuristics to gain higher coverage
 - Capa already detects GetCommandLineA usages within main
- A rule would require a tool with advanced program tracing/emulation techniques



Tools

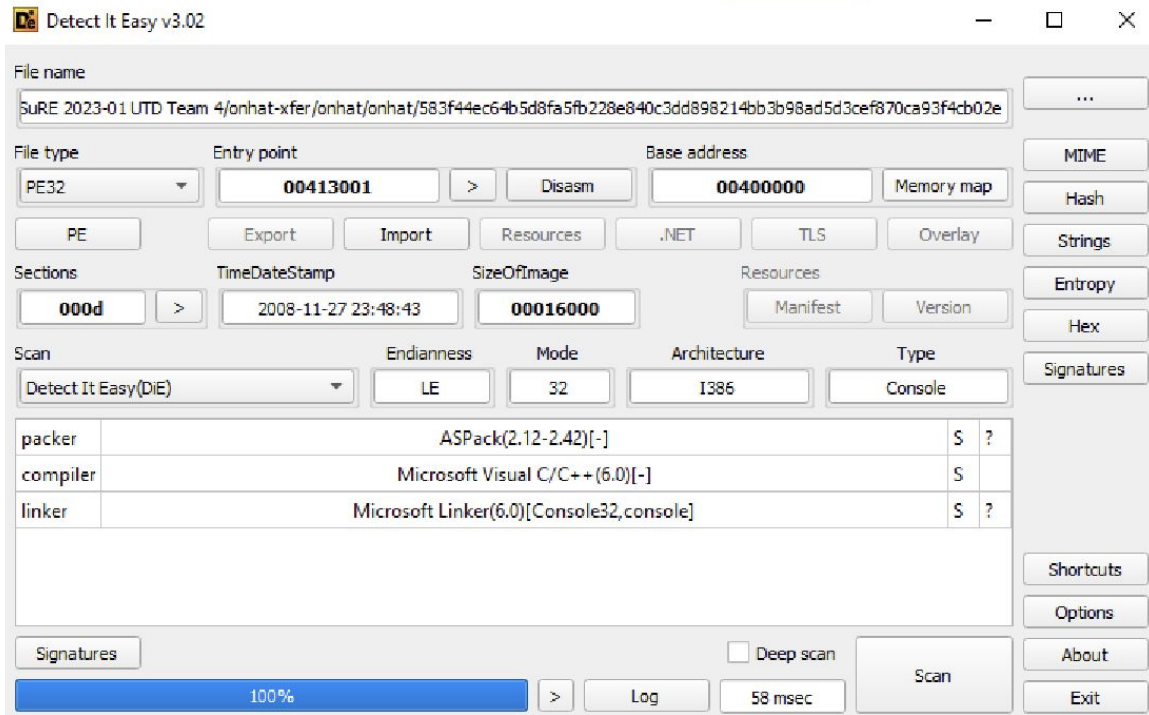
We used the following tools in our analysis:

- DetectItEasy
- Capa
- FLOSS
- Hybrid Analysis



DetectItEasy

- It is a free and open-source tool.
- Used to determine characteristics of a binary file.
- Available for Windows, Linux, and macOS.





Capa

- Developed by Mandiant.
- It is an open-source tool written in Python.
- Used to identify similar malware samples.
- Generates a set of rules to describe malware behaviour.
- Supports a range of file formats, including Windows executables, DLLs, and scripts.

Capa Rules

```
push    offset Name          ; "SADFHUHF"
push    eax                  ; bInitialOwner
push    eax                  ; lpMutexAttributes
call    ds:CreateMutexA
lea     ecx, [esp+1208h+WSAData]
push    ecx                  ; lpWSAData
push    202h                 ; wVersionRequested
call    ds:WSAStartup
test    eax, eax             Mnemonics
jnz     loc_100011E8
push    6                    ; protocol
push    1                    ; type
push    2                    ; af
call    ds:socket
mov     esi, eax
cmp     esi, 0FFFFFFFFh
jz      loc_100011E2
push    offset cp             String references
                                ; "127.26.152.13"
mov     [esp+120Ch+name.sa_family], 2
call    ds:inet_addr
```

rule:

meta:

```
name: create TCP socket
namespace: communication/socket/tcp
author: william.ballenthin@fireeye.com
```

scope: basic block ← Match context

examples:

```
- Practical Malware Analysis Lab 01-01.dll_0x10001010
```

features:

```
- and:
- number: 6 = IPPROTO_TCP
- number: 1 = SOCK_STREAM
- number: 2 = AF_INET
- or:
- api: ws2_32.socket
- api: ws2_32.WSASocket
```

} Rule logic

FLOSS (FLARE Obfuscated String Solver)



- Developed by Mandiant.
- Used for analyzing and extracting information from malware.
- Can identify indicators of compromise (IOCs).
- Can detect obfuscated and encrypted strings.

```
FLARE FLOSS RESULTS (version v2.2.0-0-g783dd8f)
-----+-----
| file path          | C:\Users\flare\Desktop\Malware Samples\INSuRE 2023-01 UTD Team 4\onhat-xfer\onhat\onhat\02f1d
e3e80954bbbe8579aa3bbba81f18634d27e05d3a41c2a9e4448cc6187f7.exe |
| extracted strings  | |
| static strings     | 462 |
| stack strings      | 17  |
| tight strings      | 0   |
| decoded strings    | 3   |
-----+-----
```

```
-----+-----
| FLOSS STACK STRINGS (17) |
-----+-----
CONNECTS SERVER ERROR[%d].
[ONHAT] CLOSSES EVENT (%d, %d).
NoHAT
nohat
nohat
nohat
NoHAT
b:ef:hl:o:O:p:P:s:t:
ONTAH_ENV_THRU_BIND
:ef:hl:o:O:p:P:s:t:
ONTAH_ENV_ONTAH
ONTAH_ENV_SERV_PORT_X
ONTAH_ENV_THRU_PORT_X
ONTAH_ENV_SERV_BIND
ONTAH_ENV_SERV_PORT
ONTAH_ENV_THRU_PORT
ONTAH_ENV_SERV_ADDR
ONTAH_ENV_THRU_ADDR

-----+-----
| FLOSS TIGHT STRINGS (0) |
-----+-----

-----+-----
| FLOSS DECODED STRINGS (3) |
-----+-----
nohat
NoHAT
NoHA
```



Hybrid Analysis

- Security tool used for automated analysis of malware samples,
- It is a malware sandbox.
- Used to capture the system-level behavior of the malware.

The screenshot displays the Hybrid Analysis web interface. At the top, there's a navigation bar with the Hybrid Analysis logo and links for Sandbox, Quick Scans, File Collections, Resources, and Request Info. The main section is titled "Analysis Overview" and contains a table of submission details. To the right of this table, there's a "malicious" badge and threat intelligence data. Below the overview, the "Extracted Strings" section is visible, featuring a search bar and a list of memory strings.

Analysis Overview	
Submission name:	file
Size:	18KiB
Type:	peexe executable
Mime:	application/x-dosexec
SHA256:	02f1de3e80054bbbe8579aa3bbba81f18634d27e05d3a41c2a9e4448cc618717
Operating System:	Windows
Last Anti-Virus Scan:	04/26/2023 06:09:57 (UTC)
Last Sandbox Report:	02/22/2023 21:18:00 (UTC)

malicious

Threat Score: 100/100
AV Detection: 73%
Labeled as: Fugra.Generic

[Link](#) [Twitter](#) [E-Mail](#)

Extracted Strings

[Download All Memory Strings \(463B\)](#)

[All Strings \(93\)](#) [Interesting \(45\)](#) [02f1de3e80054bbbe8579...](#) [screen_0.png \(7\)](#) [file.exe:3288 \(24\)](#)

- H_ENV_SERV_BIND
- INVALID FARPROC. TYPE ONTAH.EXE -h FOR HELP INFORMATION.
- INVALID PARAMETERS. TYPE ONHAT.EXE -h FOR HELP INFORMATION.
- ONTAH.EXE [-f N] LOCATION REMOTION [-b ADDR] [-a ADDR] [-p] P PORT
- ONTAH_ENb:efhlo:Op:Pst:
- ONTAH_ENV_SERV_BIND
- ONTAH_ENV_SERV_PORT
- ONTAH_ENV_THRU_BIND
- ONTAH_ENV_THRU_PORT_X
- ProviderInfo
- RV_PORT

Custom script (run_tools.sh)



```
while getopts "d:" arg; do
    case $arg in
        d) # The directory or file to run tools on
            DIR=$OPTARG;;
        esac
    done

    echo Scanning directory $DIR
    files=$(find $DIR \
        -type f \
        ! -name "*.zip" \
        ! -name "*.floss" \
        ! -name "*.capa" \
        ! -name "*.strings" \
        ! -name "*.objdump" \
        ! -name "*.txt" \
        ! -name "*.pefeat" \
        ! -name "*.err" \
        ! -name "*.chm" \
        ! -name "*.swp"
    )

    echo "hash,is_packed,pefeat,floss,capa" > data.csv

    for file in "${files[@]}"; do
        echo Processing $file

        fname=$(basename $file)

        output=$(python3 detect_packed.py $file)
        is_packed=$(echo $?)
        echo "$output" >> detect_packed.output

        if [[ $is_packed == 2 ]]
        then
            is_packed="True"
            python write_csv.py $fname $is_packed "" "" ""
        else
            is_packed="False"

            objdump_file="$file".objdump
            pefeat_file="$file".pefeat
            floss_file="$file".floss
            capa_file="$file".capa

            [ ! -f $pefeat_file ] && echo -e "\t" Running PEFeatExt && timeout 600 python3 PEFeatExt/PEFeatExt.py $file > $pefeat_file
            [ ! -f $floss_file ] && echo -e "\t" Running FLOSS && timeout 600 floss $file > $floss_file
            [ ! -f $capa_file ] && echo -e "\t" Running capa && timeout 600 capa -v $file > $capa_file

            objdump -M intel -d $file > $objdump_file
            objdump_contents=$(cat $objdump_file)

            python write_csv.py $fname $is_packed "$pefeat_file" "$floss_file" "$capa_file"
        fi
    done
done
```

data.csv



```
data.csv
1 hash,is_packed,pefeat,floss,capa
2 23873bf2670cf64c2440058130548d4e4da412dd,False,"Filename: 23873bf2670cf64c2440058130548d4e4da412dd
3 Program language not recognized. This program currently checks for Java, Python, and Go executables.
4 Programming Language: Unknown
5 Timestamp: 2012-08-09 14:56:49
6 File Type: PE32 executable (console) Intel 80386, for MS Windows
7 Machine: 64-bit (x86-x64)
8 Magic: 32-bit binary
9 Metadata:
10 Crypto:
11 Antidebug:
12     GetLastError
13     IsDebuggerPresent
14     TerminateProcess
15     UnhandledExceptionFilter
16 Antivm:
17 Packers:
18     Obsidium_v10061
19     VC8_Microsoft_Corporation
20     Microsoft_Visual_Cpp_8
21     Visual_Cpp_2008_Release_Microsoft
22 Files:
23     USER32.DLL Library
24     KERNEL32.dll Library
25     ADVAPI32.dll Library
26 URLs:
27 IPs:
```

String Extraction Script

```
commandExtract.py x
1 import re
2 import os
3
4 file = '/content/data.md'
5 strings = []
6 with open(file, 'r') as f:
7     lines = f.read().split('\n')
8     for line in lines:
9         strings.append(line)
10
11 #strings that:
12 #1. contains '-', which followed by words
13 #2. contains '\', which followed by words
14 regex1 = r'\s-[a-zA-Z]+|\\[a-zA-Z]+'
15
16 with open("Result1.txt", "w") as output:
17     output.write("Possibly command lines: " + "\n")
18     for string in strings:
19         if re.search(regex1, string):
20             output.write(string + "\n")
21
22 #strings that:
23 #1. contains words which surrounded by '%'
24 regex2 = r'\%[a-zA-Z]+\%'
25
26 with open("Result2.txt", "w") as output:
27     output.write("Possibly enviornment variables: " + "\n")
28     for string in strings:
29         if re.search(regex2, string):
30             output.write(string + "\n")
```

```
-gc Search Global Catalog (port 3268).
-p port Alternate method to specify port to connect to.
-s scope Scope of search. Base, One[Level], Sub[tree].
-t xxx Timeout value for query, default 120 seconds.
-c Object count only.
-dn Object DN's only.
-appver Output AdFind versioning info.
adfind -b dc=joehome,dc=net -f "objectcategory=computer"
adfind -b dc=joehome,dc=net -f "objectcategory=computer" cn createTimeStamp
adfind -h .:50000 -b cn=ab -f "objectcategory=person"
-h host:port Host and port to use. If not specified uses port 389 on
Port can also be specified via -p and -gc.
-gc Search Global Catalog (port 3268).
-gcb Combines -gc -null switches. i.e. Full forest search.
-gco Only use GC port; do not use port 389. Note that normal secure bind
also use -arecex and -ntlm (or -digest or -simple).
-this xxx Combines -s BASE and -b xxx
-p port Alternate method to specify port to connect to.
-hh host:port Combines -h with -arecex
-hd host:port Combines -h with -default
-writeable Use a writeable domain controller.
-kerbenc Kerberos Encryption (LDAP_OPT_ENCRYPT).
-ssl Use SSL
-sslignorecert Ignore any problems with the SSL server cert.
-starttls Use StartTLS
-arecex Hostname has a actual host name, not domain name.
-url xx Specify LDAP(S) URL.
-s scope Scope of search. Base, OneLevel, Subtree.
-base Alias for -s base.
-one Alias for -s onelevel.
-onelevel Alias for -s onelevel.
-sub Alias for -s subtree.
-subtree Alias for -s subtree.
```



Challenges

- Project Environment Setup
 - Windows and Linux Virtual environment support on Windows and M1 Mac Machines
- Limited Domain Knowledge
- Finding correct samples
- Extracting Heuristics
- Applying Heuristics



Future Work

Improving heuristics.

- Manual Identification
- Improve Tools
- Symbolic Execution

Finding ways to automate identification.

- CAPA Rules
- Yara Rules
- Machine Learning Model



Any Questions?

Thank you to INSuRE and our sponsor Frank Posluszny